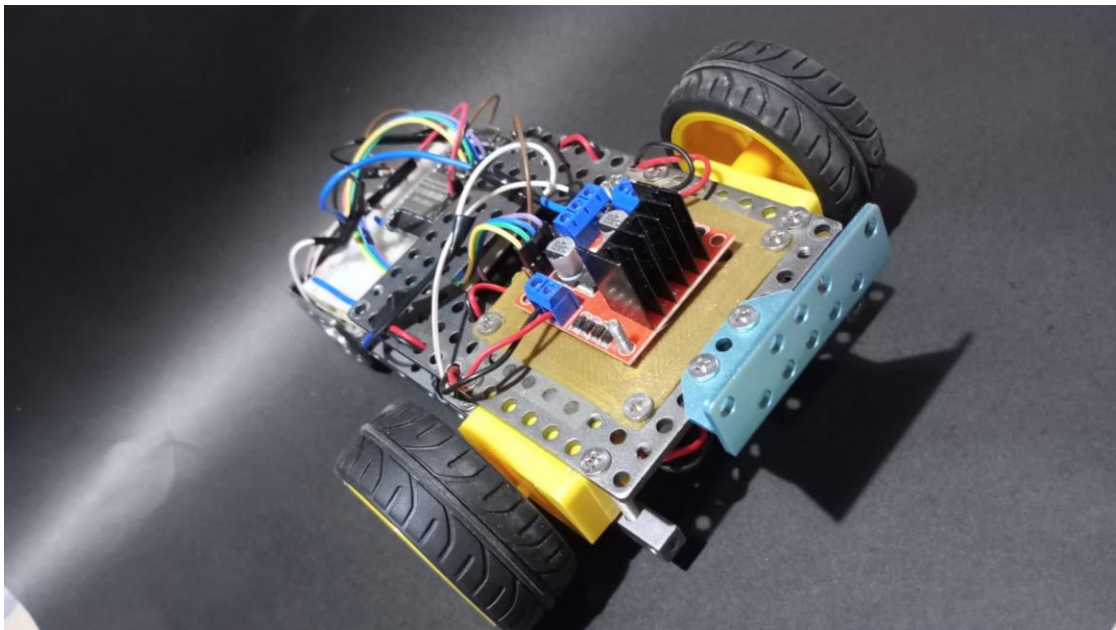


AUTO ROBOT SEGUIDOR DE LÍNEA

Informe Técnico de Proyecto



Integrantes: Roberto Estrella, Mauro Díaz, Cristian Ferrara

Electrónica General y Electrónica Digital

Docente: Dra. Diana Yelós

Año 2026

Instituto Tecnológico Universitario

1. Introducción

El presente informe describe el diseño, construcción, programación y puesta en marcha de un vehículo autónomo seguidor de línea desarrollado utilizando una placa NodeMCU ESP32, dos sensores ópticos reflectivos CNY70 y un puente H L298N para el control de motores de corriente continua.

El sistema fue diseñado para detectar una trayectoria marcada mediante cinta negra y corregir automáticamente su dirección de avance mediante un algoritmo implementado en software. Inicialmente se preparó para cinta negra sobre fondo blanco y luego se adaptó a una pista de exposición con cinta negra sobre piso azul.

Durante el desarrollo se realizaron diversas etapas de diseño mecánico, integración electrónica, calibración de sensores y optimización del algoritmo de control hasta obtener un vehículo capaz de completar repetidamente una pista de ensayo de forma autónoma.

2. Objetivos

2.1 Objetivo general

Diseñar e implementar un vehículo autónomo capaz de seguir una trayectoria marcada mediante sensores ópticos y control diferencial de motores.

2.2 Objetivos específicos

- Implementar un sistema de control basado en ESP32.
- Utilizar sensores CNY70 para la detección de línea.
- Controlar dos motores DC mediante un puente H L298N.
- Realizar ensayos de calibración y ajuste de umbrales.
- Desarrollar un algoritmo de corrección de trayectoria en tiempo real.
- Validar el funcionamiento del sistema sobre una pista real.

3. Desarrollo y evolución del prototipo

El proyecto comenzó a partir de un chasis metálico perforado equipado con dos motores de corriente continua con caja reductora y ruedas motrices. En la primera configuración los motores se encontraban montados en posiciones desfasadas longitudinalmente, formando una plataforma mecánica inicial para la posterior incorporación de electrónica, sensores y alimentación.

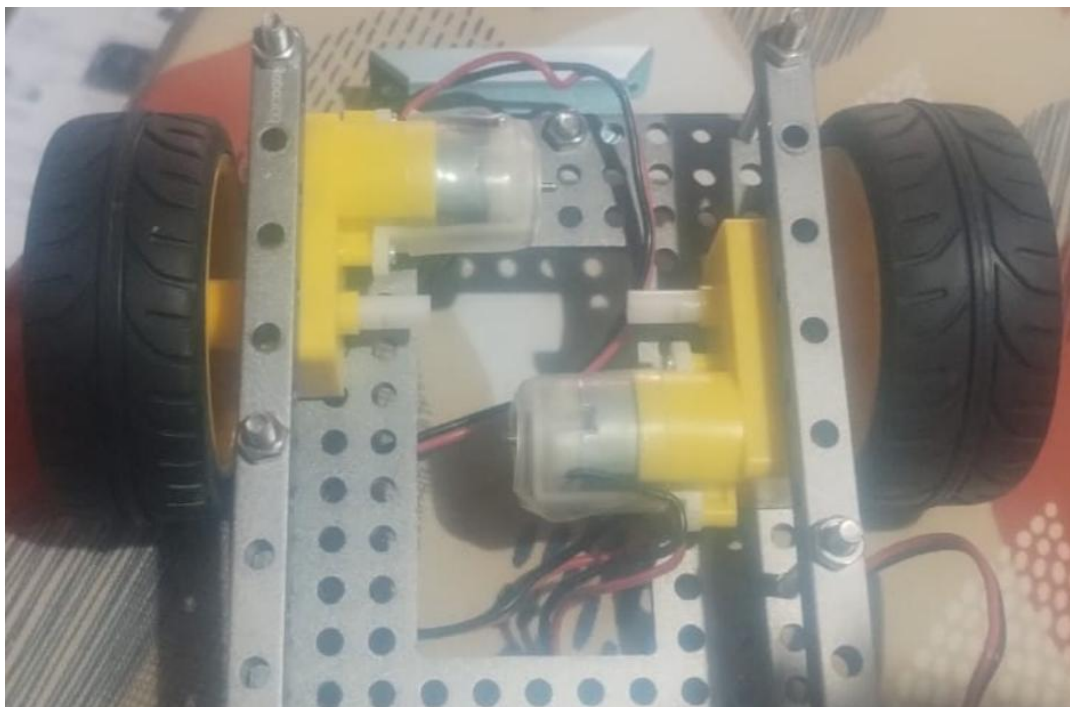


Figura 1. Prototipo mecánico inicial con motores desfasados longitudinalmente.

Desde las primeras etapas se definió la ubicación de los sensores reflectivos CNY70 en la parte frontal. Considerando que la pista estaría formada por una cinta negra de aproximadamente 48 mm de ancho, se estableció una separación de 30 mm entre sensores. Esta disposición permite que ambos sensores permanezcan simultáneamente sobre la línea cuando el vehículo está centrado.



Figura 2. Montaje frontal de sensores CNY70 sobre soporte metálico.

La versión final integra en un único prototipo el sistema mecánico, la etapa de sensado, la unidad de control, la etapa de potencia y el sistema de alimentación.

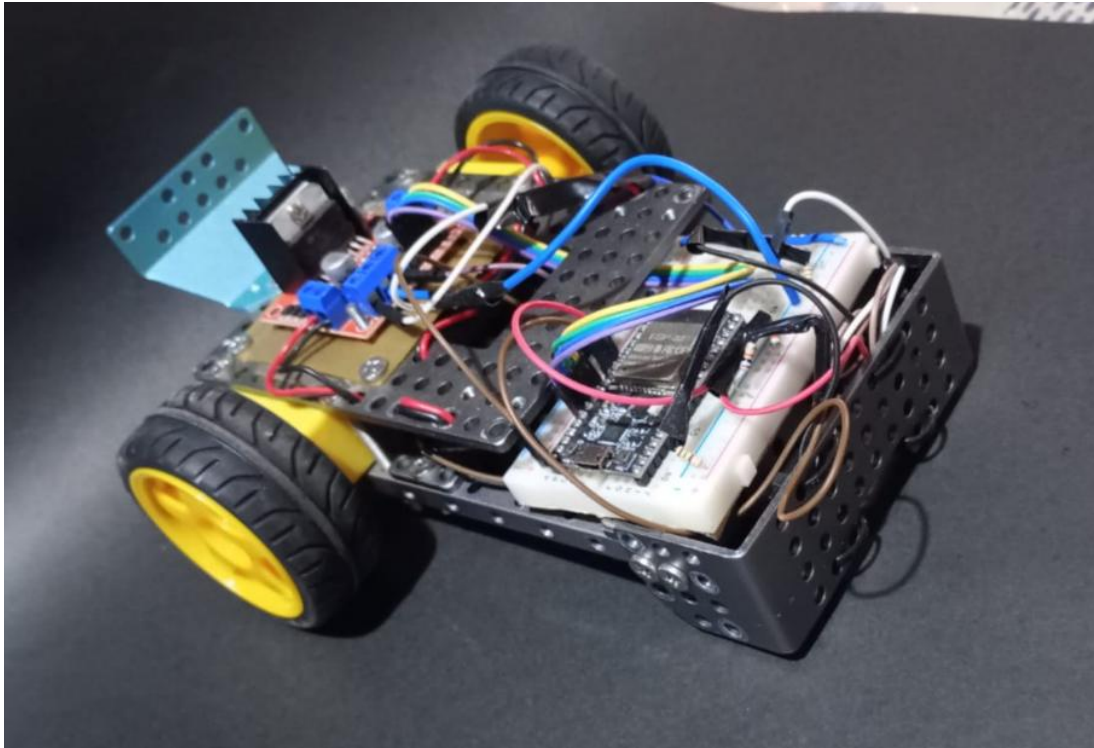


Figura 3. Vista diagonal frontal de la versión final del Auto Robot.

4. Diseño mecánico

Parámetro	Valor aproximado
Largo total	210 mm
Largo sin alerón trasero	180 mm
Ancho total con ruedas	165 mm
Ancho del chasis	90 mm
Altura máxima	75 mm

El sistema de locomoción se basa en tracción diferencial mediante dos ruedas motrices accionadas por motores DC con caja reductora. La rueda guía frontal está formada por un rodillo cilíndrico afinado hacia los extremos, fijado al chasis mediante dos tornillos laterales con tuercas.



Figura 4. Rueda guía frontal fijada con tornillos laterales.

En la parte posterior se incorporó un alerón metálico de función estética y de sujeción, útil para manipular el vehículo y colocarlo sobre la pista durante las pruebas.

5. Sistema electrónico

5.1 Unidad de control

La unidad de procesamiento utilizada corresponde a una placa NodeMCU ESP32 de 38 pines basada en el microcontrolador ESP-32S. La placa cuenta con conexión micro USB, conectividad WiFi y Bluetooth, pines de entrada/salida digitales y entradas analógicas empleadas para la lectura de los sensores.

5.2 Sensores CNY70

Para la detección de línea se utilizaron dos sensores ópticos reflectivos CNY70. Cada sensor integra un LED infrarrojo emisor y un fototransistor receptor. La configuración eléctrica utilizada fue: resistencia de 150 ohm para el LED IR y resistencia pull-up de 10 kOhm para el fototransistor.

Los sensores quedaron ubicados 10 mm por delante de la rueda guía, separados 30 mm entre sí y a una altura aproximada de 3 mm respecto del piso.

5.3 Sistema de potencia

El control de motores se realizó mediante un módulo puente H L298N. Este módulo permite controlar de forma independiente el sentido de giro y la velocidad de cada motor mediante señales digitales y PWM.

Para el montaje del módulo L298N se utilizó una base fabricada mediante impresión 3D, lo que permitió fijarlo mecánicamente al chasis y ordenar la distribución del conjunto.

5.4 Alimentación

El sistema se alimenta mediante un portapilas de 6 pilas AA Ni-MH recargables de 2000 mAh. El portapilas se fijó en el piso del chasis mediante tornillería de abajo hacia arriba.

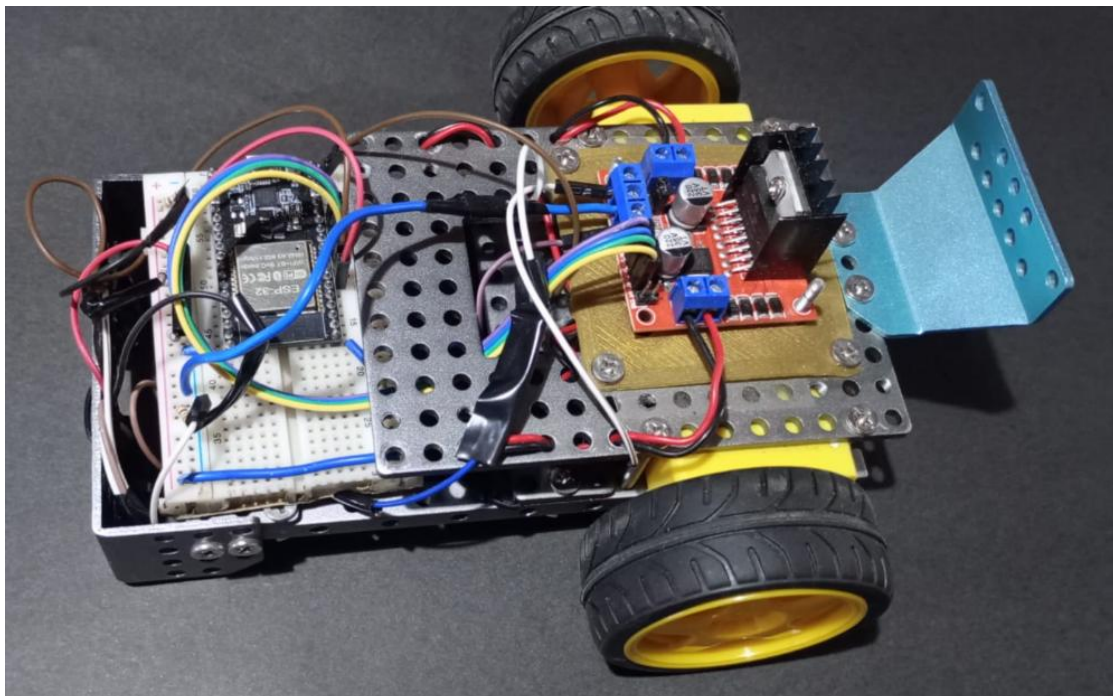


Figura 5. Vista superior de la arquitectura electrónica: ESP32, protoboard, cableado y L298N.

6. Asignación de pines

Bloque	Función	GPIO ESP32
Sensor izquierdo	Entrada analógica ADC	32
Sensor derecho	Entrada analógica ADC	33
Motor izquierdo	IN1	12
Motor izquierdo	IN2	14
Motor izquierdo	ENA / PWM	25
Motor derecho	IN3	27
Motor derecho	IN4	26
Motor derecho	ENB / PWM	13

7. Calibración experimental

El robot fue preparado inicialmente para trabajar con cinta negra sobre fondo blanco. Luego, para la exposición, se adaptó a cinta negra sobre piso azul, lo que obligó a recalibrar el umbral de detección por las diferencias de reflectancia y de iluminación del lugar.

Superficie	Lectura ADC típica
Blanco	24, 60 y 120
Piso azul	~1700
Piso azul con mayor reflectancia	~2100
Cinta negra	2420 - 2460
Umbral final utilizado	2300

A partir de las mediciones se estableció un umbral de 2300. En la versión funcional del programa, la línea negra se interpreta como una lectura superior al umbral.

8. Algoritmo de control

El programa realiza lecturas analógicas de ambos sensores y calcula un promedio de cinco muestras para reducir fluctuaciones. Luego transforma esas lecturas en estados lógicos: sensor sobre negro o sensor fuera de la línea.

Sensor izquierdo	Sensor derecho	Acción del robot
Negro	Negro	Avance recto
Negro	Blanco	Corrección hacia izquierda
Blanco	Negro	Corrección hacia derecha
Blanco	Blanco	Búsqueda de línea según última corrección

La corrección se realiza variando la velocidad relativa de los motores. En recta se utiliza un PWM de 220. Para curvas, una rueda gira más rápido que la otra, evitando detenciones bruscas y mejorando la continuidad del movimiento.

8.1 Recuperación de línea

Una mejora importante fue la incorporación de memoria de última corrección. Cuando ambos sensores pierden simultáneamente la línea, el robot ejecuta una maniobra de búsqueda hacia el lado donde se había detectado la última desviación. Esta estrategia fue clave para mejorar el desempeño en curvas.

9. Problemas encontrados y soluciones implementadas

Identificación y conexionado de sensores

Fue necesario verificar la correcta identificación de terminales del LED infrarrojo y del fototransistor de los sensores CNY70.

Ajuste de velocidades

Se realizaron múltiples pruebas para determinar velocidades adecuadas tanto en rectas como en curvas.

Corrección en curvas

El principal desafío fue detectar por qué la lógica de corrección no recuperaba adecuadamente la línea en curvas. La solución consistió en implementar velocidades diferenciadas y búsqueda basada en la última corrección.

Adaptación al piso azul

La superficie azul de la exposición presentó valores más cercanos a la cinta negra que un fondo blanco, por lo que fue necesario recalibrar el umbral de detección.

10. Resultados obtenidos

- El vehículo completa la pista de ensayo de forma autónoma.
- El recorrido puede repetirse varias veces hasta el agotamiento progresivo de las baterías.
- El sistema quedó preparado para recalibración rápida en función de las condiciones de luz del lugar.
- El valor PWM de avance en recta utilizado en la versión final fue 220.

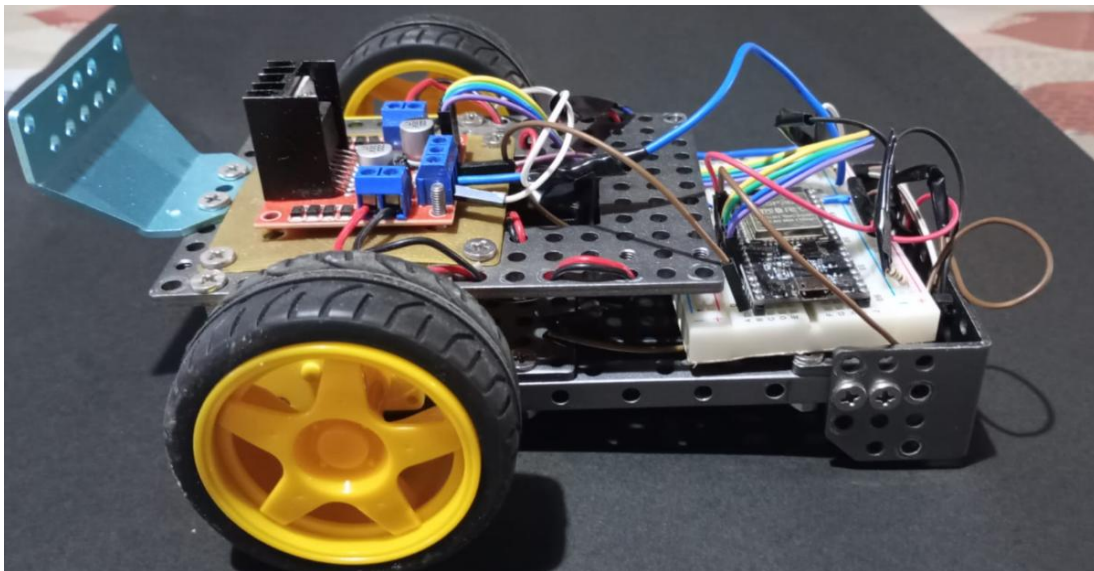


Figura 6. Vista lateral del prototipo final.

11. Conclusiones

El proyecto permitió integrar conocimientos de electrónica digital, sensores, programación, sistemas de potencia y diseño mecánico en una aplicación práctica de robótica móvil.

La experiencia demostró que el funcionamiento correcto del vehículo no depende únicamente del montaje de los componentes, sino de la integración entre diseño mecánico, lectura confiable de sensores, calibración experimental y lógica de control.

El principal desafío técnico estuvo asociado a la optimización de la lógica de corrección en curvas y a la calibración del umbral para el piso azul de exposición. Finalmente se obtuvo un Auto Robot Seguidor de Línea capaz de recorrer la pista de manera confiable y repetitiva.

Anexo A - Código fuente completo

A continuación, se presenta el programa final cargado en la placa ESP32 para el funcionamiento del Auto Robot Seguidor de Línea.

```
// PINES DE SENSORES
const int pinSensorIzquierdo = 32;
const int pinSensorDerecho = 33;

// MOTOR IZQUIERDO
const int pinIN1 = 12;
const int pinIN2 = 14;
const int pinENA = 25;

// MOTOR DERECHO
const int pinIN3 = 27;
const int pinIN4 = 26;
const int pinENB = 13;

// CONFIGURACIÓN
const int UMBRAL_NEGRO = 2300;

// Recta
const int VELOCIDAD_RECTA = 220;

// Curva fluida
const int VELOCIDAD_RAPIDA = 230;
const int VELOCIDAD_LENTA = 120;

// Búsqueda agresiva
const int VELOCIDAD_BUSQUEDA_AVANCE = 170;
const int VELOCIDAD_BUSQUEDA_REVERSA = -60;

const int TIEMPO_BUSQUEDA = 500;

int ultimaCorreccion = 0;
unsigned long tiempoPerdida = 0;

void setup() {
  Serial.begin(115200);

  pinMode(pinSensorIzquierdo, INPUT);
  pinMode(pinSensorDerecho, INPUT);

  pinMode(pinIN1, OUTPUT);
  pinMode(pinIN2, OUTPUT);
  pinMode(pinENA, OUTPUT);

  pinMode(pinIN3, OUTPUT);
  pinMode(pinIN4, OUTPUT);
  pinMode(pinENB, OUTPUT);

  ledcAttach(pinENA, 5000, 8);
  ledcAttach(pinENB, 5000, 8);

  Serial.println("Seguidor de linea listo");
}

void loop() {
  int promedioIzquierdo = readAdcAverage(pinSensorIzquierdo, 5);
  int promedioDerecho = readAdcAverage(pinSensorDerecho, 5);

  // En tu código funcional:
  // NEGRO = lectura mayor al umbral
  bool lineaIzq = (promedioIzquierdo > UMBRAL_NEGRO);
  bool lineaDer = (promedioDerecho > UMBRAL_NEGRO);

  // AMBOS SOBRE NEGRO -> RECTA
  if (lineaIzq && lineaDer) {
    controlMotores(VELOCIDAD_RECTA, VELOCIDAD_RECTA);
    ultimaCorreccion = 0;
  }

  // SENSOR DERECHO FUERA -> corregir hacia izquierda
  else if (lineaIzq && !lineaDer) {
    // Curva fluida: izquierda lenta, derecha rápida
    controlMotores(VELOCIDAD_RAPIDA, VELOCIDAD_LENTA);

    ultimaCorreccion = 1;
    tiempoPerdida = millis();
  }
}
```

```

}

// SENSOR IZQUIERDO FUERA -> corregir hacia derecha
else if (!lineaIzq && lineaDer) {
    // Curva fluida: izquierda rápida, derecha lenta
    controlMotores(VELOCIDAD_LENTA, VELOCIDAD_RAPIDA);

    ultimaCorreccion = -1;
    tiempoPerdida = millis();
}

// AMBOS FUERA -> búsqueda agresiva
else {
    if (millis() - tiempoPerdida < TIEMPO_BUSQUEDA) {
        if (ultimaCorreccion == 1) {
            controlMotores(VELOCIDAD_BUSQUEDA_REVERSA, VELOCIDAD_BUSQUEDA_AVANCE);
        }
        else if (ultimaCorreccion == -1) {
            controlMotores(VELOCIDAD_BUSQUEDA_AVANCE, VELOCIDAD_BUSQUEDA_REVERSA);
        }
        else {
            controlMotores(0, 0);
        }
    }
    else {
        controlMotores(0, 0);
    }
}

delay(2);
}

int readAdcAverage(int pin, int muestras) {
    long suma = 0;

    for (int i = 0; i < muestras; i++) {
        suma += analogRead(pin);
        delayMicroseconds(4);
    }

    return suma / muestras;
}

void controlMotores(int velocidadIzquierda, int velocidadDerecha) {
    if (velocidadIzquierda >= 0) {
        digitalWrite(pinIN1, HIGH);
        digitalWrite(pinIN2, LOW);
    } else {
        digitalWrite(pinIN1, LOW);
        digitalWrite(pinIN2, HIGH);
        velocidadIzquierda = -velocidadIzquierda;
    }

    if (velocidadDerecha >= 0) {
        digitalWrite(pinIN3, HIGH);
        digitalWrite(pinIN4, LOW);
    } else {
        digitalWrite(pinIN3, LOW);
        digitalWrite(pinIN4, HIGH);
        velocidadDerecha = -velocidadDerecha;
    }

    ledcWrite(pinENA, velocidadIzquierda);
    ledcWrite(pinENB, velocidadDerecha);
}

```